Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Lab on Authentication

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

Authentication Fundamentals

> Authentication = the process of reliably verifying identity (user or system).

Authentication Fundamentals

> Authentication = the process of reliably verifying identity (user or system).

Authentication differs depending on who must be authenticated:

Authentication Fundamentals

> Authentication = the process of reliably verifying identity (user or system).

Authentication differs depending on who must be authenticated:

A) **Computers** can:

- store high-quality secrets (long random numbers).
- perform cryptographic operations.

Authentication Fundamentals

> Authentication = the process of reliably verifying identity (user or system).

Authentication differs depending on who must be authenticated:

A) **Computers** can:

- store high-quality secrets (long random numbers).
- perform cryptographic operations.

B) **Humans** can only remember weak secrets (passwords), which are used to:

- Derive a cryptographic key (e.g., hash of password).
- Decrypt a higher quality key locally (e.g., decrypt RSA key).

Authentication Factors

Authentication mechanisms fall into four categories:

1. **Knowledge** (What you know)
   - Includes: secrets users remember (passwords, PINs, security questions).
   - Weakness: vulnerable to guessing, reuse, and phishing; Limited by human memory.

2. **Possession** (What you have)
   - Includes: hardware tokens, One-Time Passwords (OTP) apps, physical devices performing crypto operations.
   - Weakness: device loss or theft; SMS containing OTP can be intercepted.

3. **Inherence** (Who you are)
   - Includes: biometric traits (fingerprint, face, voice, retina).
   - Weakness: not secret, hard to revoke, spoofable depending on implementation.

4. **Location** (Where you are)
   - Includes: physical or network location, contextual signals.
   - Weakness: easily spoofed; useful only as an additional risk signal.

Authentication Factors

Authentication mechanisms fall into four categories:

1. **Knowledge** (What you know)
   - Includes: secrets users remember (passwords, PINs, security questions).
   - Weakness: vulnerable to guessing, reuse, and phishing; Limited by human memory.

2. **Possession** (What you have)
   - Includes: hardware tokens, One-Time Passwords (OTP) apps, physical devices performing crypto operations.
   - Weakness: device loss or theft; SMS containing OTP can be intercepted.

3. **Inherence** (Who you are)
   - Includes: biometric traits (fingerprint, face, voice, retina).
   - Weakness: not secret, hard to revoke, spoofable depending on implementation.

4. **Location** (Where you are)
   - Includes: physical or network location, contextual signals.
   - Weakness: easily spoofed; useful only as an additional risk signal.

NOTE: Security increases by combining independent factors → **Multi-Factor Authentication** (MFA)

Password-Based Authentication

Password Challenges

Common problems:

1. **Robustness:** Human-chosen passwords are weak (short, not random).
2. **Transmission:** Never send passwords in clear (vulnerable to sniffing).
3. **Storage:** Must store passwords securely (encrypted/hashed).
4. **Reuse**: Users often reuse passwords across multiple sites (if leaked, all accounts are compromised).

Password Challenges

Common problems:

1. **Robustness:** Human-chosen passwords are weak (short, not random).
2. **Transmission:** Never send passwords in clear (vulnerable to sniffing).
3. **Storage:** Must store passwords securely (encrypted/hashed).
4. **Reuse**: Users often reuse passwords across multiple sites (if leaked, all accounts are compromised).

> **Takeaway**: passwords alone are fragile; Proper storage and policies are essential.

How a website/application should store a password?

Different approaches:

How a website/application should store a password?

Different approaches:

1. *Plain text*:
   - The password is stored as-is in the database.
   - If the database is leaked, the attacker has access to all clear passwords.

How a website/application should store a password?

Different approaches:

1. *Plain text*:
   - The password is stored as-is in the database.
   - If the database is leaked, the attacker has access to all clear passwords.

2. *Hashed*:
   - The password is stored as $\mathrm{hash}(password)$.
   - The website verifies if a password is correct by checking if $\mathrm{hash}(entered_password) = \mathrm{hash}(stored_password)$.
   - If the database is leaked, the attacker gets all hashed passwords, but not the original ones.

How a website/application should store a password?

Different approaches:

1. *Plain text*:
   - The password is stored as-is in the database.
   - If the database is leaked, the attacker has access to all clear passwords.

2. *Hashed*:
   - The password is stored as $\mathrm{hash}(password)$.
   - The website verifies if a password is correct by checking if $\mathrm{hash}(entered_password) = \mathrm{hash}(stored_password)$.
   - If the database is leaked, the attacker gets all hashed passwords, but not the original ones.

3. *Salted* and *Hashed*:
   - The password is stored as $hash(password \,||\, salt)$.
   - salt = unique random value that is stored with the password in the database.
   - The verification process is the same as above.

How a website/application should store a password?

Different approaches:

1. *Plain text*:
   - The password is stored as-is in the database.
   - If the database is leaked, the attacker has access to all clear passwords.

2. *Hashed*:
   - The password is stored as $\mathrm{hash}(password)$.
   - The website verifies if a password is correct by checking if $\mathrm{hash}(entered_password) = \mathrm{hash}(stored_password)$.
   - If the database is leaked, the attacker gets all hashed passwords, but not the original ones.

3. *Salted* and *Hashed*:
   - The password is stored as $hash(password \,||\, salt)$.
   - salt = unique random value that is stored with the password in the database.
   - The verification process is the same as above.

How can we break a hashed password?

Password Attacks

There are three main categories of attacks against passwords:

1. **Brute Force Attack**
2. **Dictionary Attack**
3. **Rainbow Tables**

Each has different characteristics, costs, and countermeasures.

Brute Force Attack: Definition

Exhaustive key search, i.e., we try all possible combinations systematically.

Characteristics:

- Guaranteed to find the password eventually.
- Time depends on password space size.
- No intelligence or optimization.
- Works against any password (weak or strong).

Brute Force Attack: Definition

Exhaustive key search, i.e., we try all possible combinations systematically.

Characteristics:

- Guaranteed to find the password eventually.
- Time depends on password space size.
- No intelligence or optimization.
- Works against any password (weak or strong).

EXAMPLE: For a 4-digit PIN:

- Password space: $10^4 = 10,000$ combinations.
- Try: 0000, 0001, 0002, ..., 9999.
- Average attempts to find password: $10,000/2 = 5,000$.

Brute Force: Password Space

The size of the password space depends on:

- Character set size (alphabet).
- Password length.

$$\text{Password Space} = |\text{Alphabet}|^{\text{Length}}$$

Brute Force: Password Space

The size of the password space depends on:

- Character set size (alphabet).
- Password length.

$$\text{Password Space} = |\text{Alphabet}|^{\text{Length}}$$

Common character sets:

Character Set	Size	Example
Digits only	10	0-9
Lowercase letters	26	a-z
Upper + lowercase	52	a-z, A-Z
Alphanumeric	62	a-z, A-Z, 0-9
Alphanumeric + symbols	94	a-z, A-Z, 0-9, !@#$...

Brute Force: Time Estimates

Assuming 1 billion (10^9) attempts per second:

Length	Lowercase (26)	Alphanumeric (62)	Full ASCII (94)
4	< 1 second	< 1 second	< 1 second
6	< 1 second	57 seconds	34 minutes
8	5 minutes	2.4 days	60 days
10	3.5 hours	9.5 years	1,500 years
12	100 days	59,000 years	13 million years

Brute Force: Python

> The core pattern of a brute force attack is a nested loop over the character space (one loop per character).

```python
alphabet = "abcdefghijklmnopqrstuvwxyz"  # a-z

# assuming the target has length 2
for c1 in alphabet:
    for c2 in alphabet:
        guess = c1 + c2
        ...   # submit guess to system you're trying to break into
```

Possible alphabets:

- Digits only: "0123456789"
- Lowercase letters: "abcdefghijklmnopqrstuvwxyz"
- Upper + lowercase: "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
- Alphanumeric: "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
- Alphanumeric + symbols: "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$..."

Brute Force: Python

> You had already seen an example of brute forcing in the very first lab!

```python
# Ceasar Cipher brute force:

def caesar_shift(text, shift):
    result = ""
    for char in text:
        ...

ciphertext = "..."

for shift in range(26):
    caesar_shift(ciphertext, shift)
```

Brute Force: Python

> Sometimes you may know something about the output → use it to stop early.

```python
# Text to match: "Password: ..."
# You know that it starts with Password:

known_text = "..."

# brute-forcing of possible single-bit key values:

for key in range(256):
    result = ""
    for n in known_text:
        result += chr(n ^ key)
    if result.startswith("Password:"):
        print(result)
        break
```

Dictionary Attack: Definition

Dictionary Attack: Definition

A *smart* brute force where we try only likely passwords from a *dictionary*. Indeed:

Dictionary Attack: Definition

A *smart* brute force where we try only likely passwords from a *dictionary*. Indeed:

A) Humans choose predictable passwords:

- Common words: "password", "admin", "welcome".
- Names: "john", "maria", "alice".
- Dates: "2024", "123456".
- Keyboard patterns: "qwerty", "asdf".

Dictionary Attack: Definition

A *smart* brute force where we try only likely passwords from a *dictionary*. Indeed:

A) Humans choose predictable passwords:

- Common words: "password", "admin", "welcome".
- Names: "john", "maria", "alice".
- Dates: "2024", "123456".
- Keyboard patterns: "qwerty", "asdf".

B) The search becomes sustainable:

- Much smaller search space.
- Higher success rate per attempt.
- Can crack weak passwords in seconds.

Where do dictionaries come from?

Where do dictionaries come from?

1. **Common password lists:**
   - RockYou breach (32 million passwords).
   - LinkedIn breach (117 million passwords).
   - Have I Been Pwned database (billions of passwords).

Where do dictionaries come from?

1. **Common password lists:**
   - RockYou breach (32 million passwords).
   - LinkedIn breach (117 million passwords).
   - Have I Been Pwned database (billions of passwords).

2. **Language dictionaries:**
   - English, Spanish, Chinese words.
   - Common phrases and expressions.

Where do dictionaries come from?

1. **Common password lists:**
   - RockYou breach (32 million passwords).
   - LinkedIn breach (117 million passwords).
   - Have I Been Pwned database (billions of passwords).

2. **Language dictionaries:**
   - English, Spanish, Chinese words.
   - Common phrases and expressions.

3. **Targeted dictionaries:**
   - Company-specific terms.
   - Industry jargon.
   - Personal information (names, dates).

Dictionary Attack: Variations

Attackers apply **rules** to dictionary words:

Dictionary Attack: Variations

Attackers apply **rules** to dictionary words:

- **Common transformations:**
  - Capitalization: password → Password, PASSWORD.
  - Leetspeak: password → p@ssw0rd, pa55word.
  - Append numbers: password → password1, password123.
  - Append symbols: password → password!, password@.
  - Combinations: password → P@ssw0rd123!.

Dictionary Attack: Variations

Attackers apply **rules** to dictionary words:

- **Common transformations:**
  - Capitalization: password → Password, PASSWORD.
  - Leetspeak: password → p@ssw0rd, pa55word.
  - Append numbers: password → password1, password123.
  - Append symbols: password → password!, password@.
  - Combinations: password → P@ssw0rd123!.
- **Hybrid attack:** Dictionary + brute force
  - Use dictionary words as base.
  - Brute force additional characters.
  - Example: "password" + 2 digits = 100 variations.

Dictionary Attack: Python

The core pattern of a dictionary attack is a loop over the possible passwords:

```
dictionary = ["password", "admin", "welcome", "123456"]

for word in dictionary:
    ...    # submit guess to system you're trying to break into
```

In pratice, dictionaries are often loaded from files:

```
with open("rockyou.txt") as f:
    dictionary = f.read().splitlines()

for word in dictionary:
     ...    # submit guess to system you're trying to break into

# Alternative

with open("rockyou.txt") as f:
    for word in f.read().splitlines()
        ... # submit guess to system you're trying to break into
```

Dictionary Attack Variations: Python

Upper-Lower:

```python
with open("wordlist.txt") as f:
    for word in f.readlines():
        if word.lower() == target:             # word.upper() for uppercase
            print(f"Found: {word.lower()}")
```

Leetspeak:

```python
with open("wordlist.txt") as f:
    for word in f.read().splitlines():
        variation = word
        variation = variation.replace("a", "4")
        variation = variation.replace("e", "3")
        variation = variation.replace("i", "1")
        variation = variation.replace("o", "0")
        variation = variation.replace("s", "5")
        variation = variation.replace("t", "7")
        if variation == target:
            print(f"Found: {variation}")
```

Dictionary Attack Variations: Python

Capitalize:

```python
with open("wordlist.txt") as f:
    for word in f.readlines():
        if word.capitalize() == target:
            print(f"Found: {word.capitalize()}")
```

Hashed target string:

```python
import hashlib

target_hash = "..."

with open("wordlist.txt") as f:
    for word in f.read().splitlines():
        if hashlib.sha256(word.encode()).hexdigest() == target_hash:
            print(f"Found: {word}")
```

Time to Cook

theXORacle

Auth Challenge 01

theXORacle - Reconnaissance

theXORacle (500 points) auth

Surely unbreakable...unless you try hard enough.

N.B. The secret message starts with "Key:"

https://auth01.webhack.it

Flag

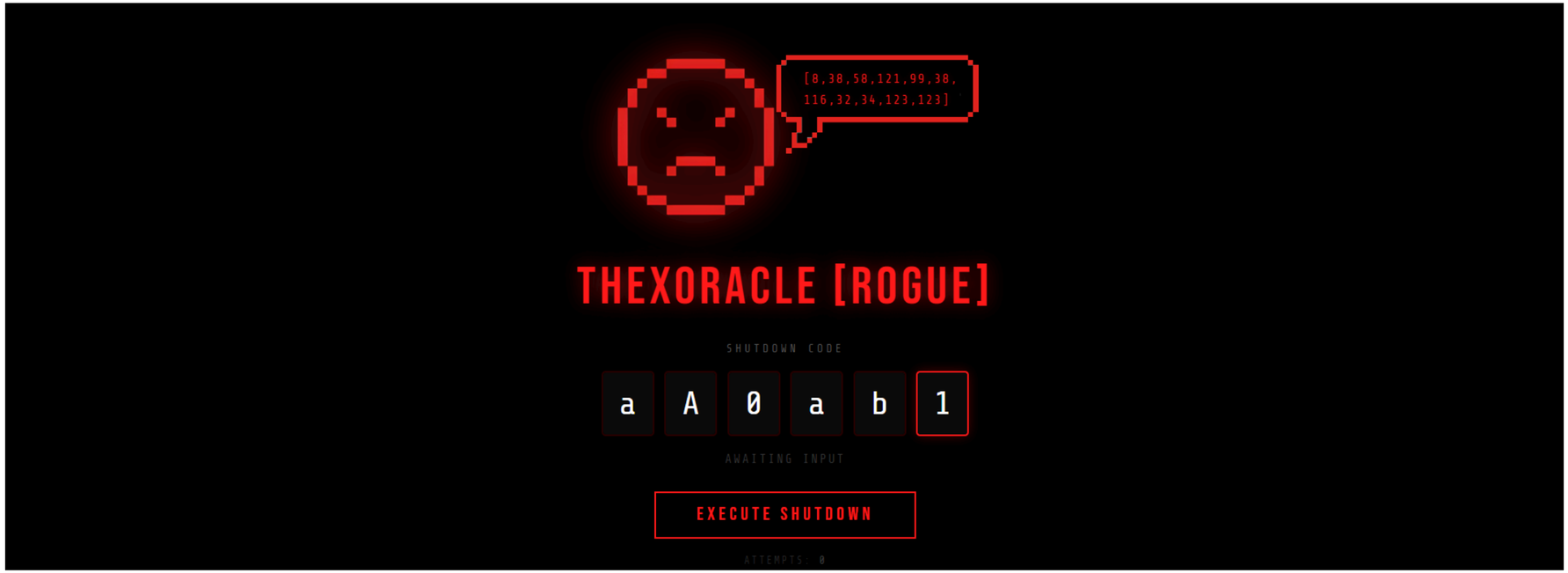
Submit

What can we observe? The challenge...

- tells us that there's a secret message to break, and that it starts with "Key:"
- displays an **encoded** message in a speech bubble.
- asks for a shutdown code to turn off a rogue AI.

theXORacle - Hypothesis

The name of the challenge hints at the XOR operation being used to scramble the shutdown code.



The scrambled message is composed of integers, BUT the shutdown code accepts alphanumeric characters...

theXORacle - Exploitation

1. **Retrieve scrambled data** from the speech bubble.

2. **Brute-force the XOR key**: try all 256 possible byte values, looking for outputs starting with "Key"...

```
result = ''.join(chr(n ^ key) for n in scrambled)
if result.startswith("Key: ")
    ...
```

3. **Extract the shutdown code**: the recovered plaintext reveals "Key: XXXXXX"

4. **Submit the shutdown code** to retrieve the flag.

theXORacle - Solution Template

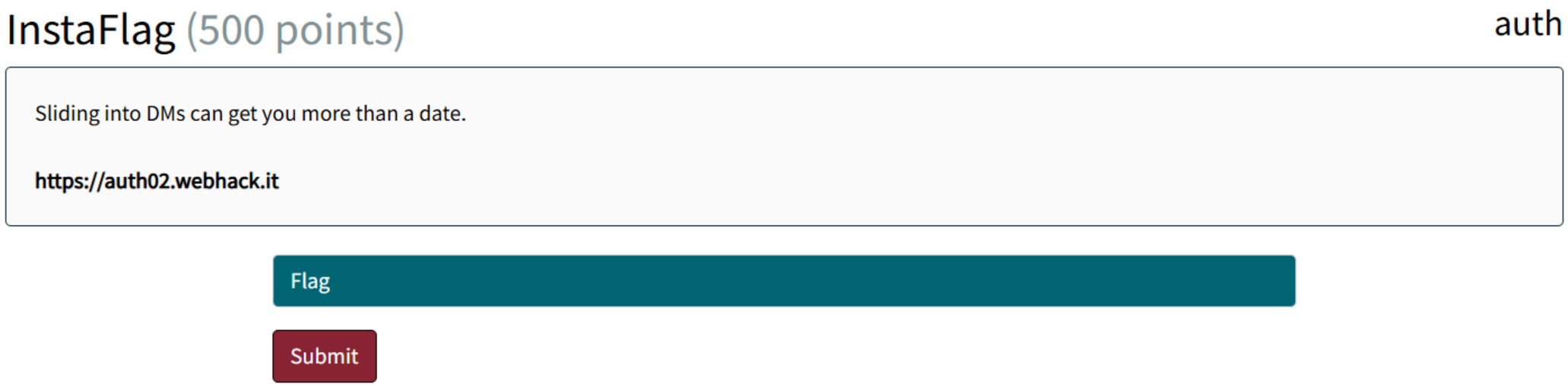
```python
scrambled = "..."  #put the scrambled data here

for key in range(256):
    result = ""
    for n in scrambled:
        result += chr(n ^ key)
    if result.startswith("Key:"):
        print(result)
```

InstaFlag

Auth Challenge 02

InstaFlag - Reconnaissance

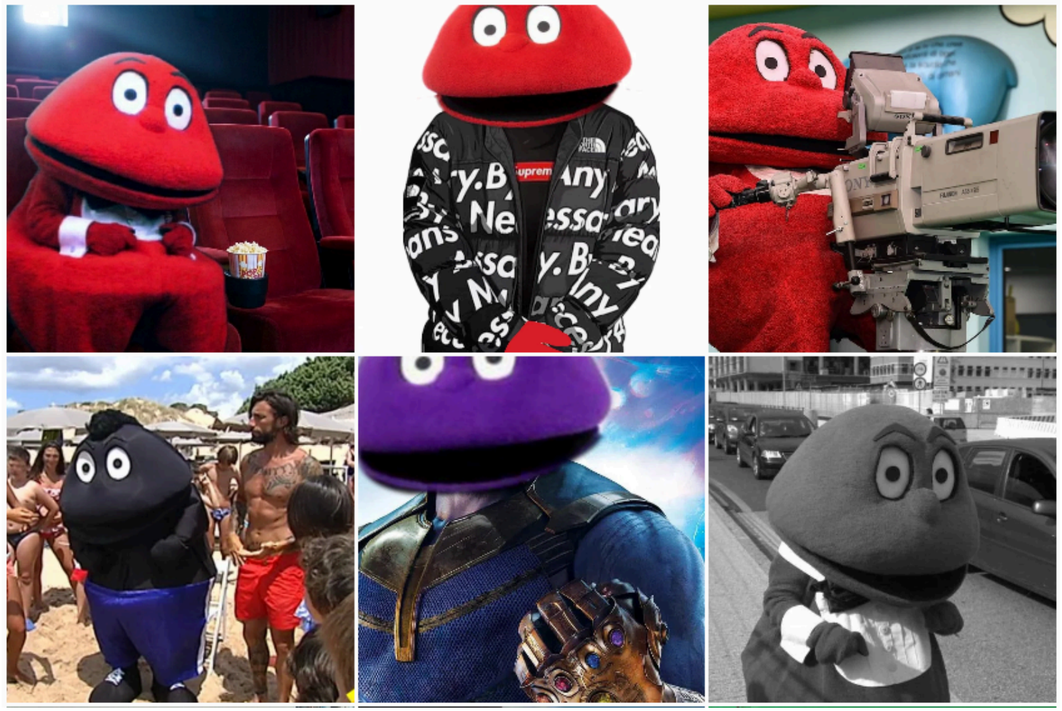


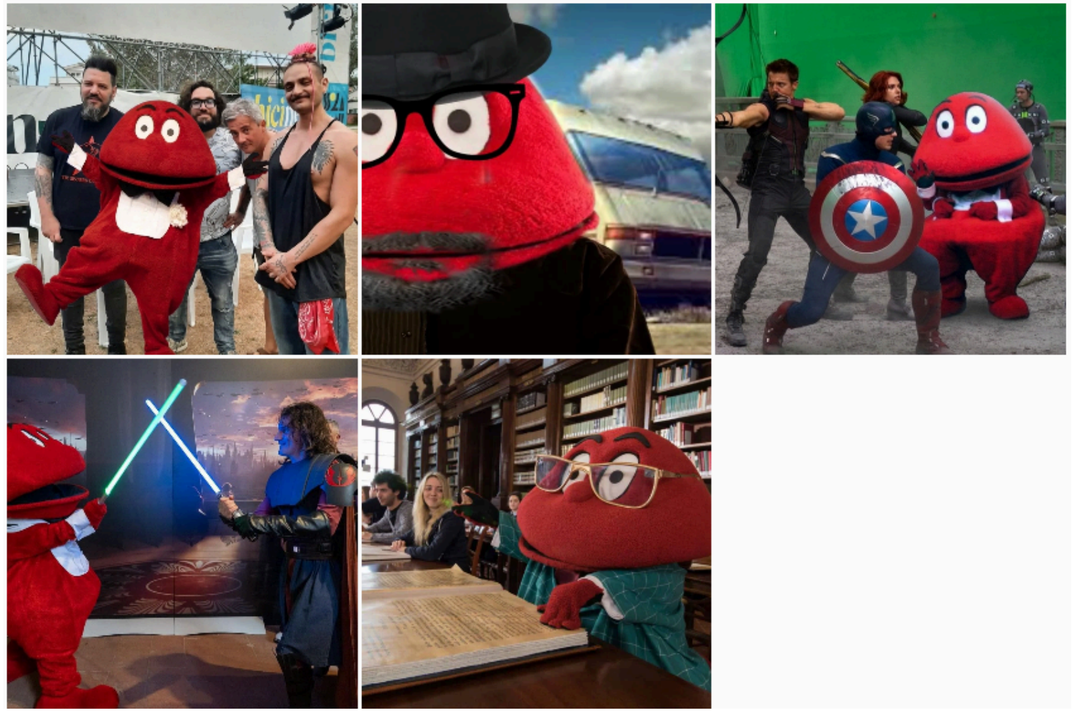
What can we observe? The challenge...

- provides an instagram-themed web interface, with direct messages and user profile.
- allows us only to visit Gabbibo's user profile.
- contains *intriguing* messages from Prof. Ralli, including an hashed password and wordlists to download.

InstaFlag - Hypothesis

The majority of Gabibbo's pictures are tv-shows/movies-related.





One of the wordlists appears to be filled with passwords referencing movies and tv-shows...

InstaFlag - Exploitation

1. **Retrieve the SHA-256 hash** of Gabbibo's password from the direct messages.

2. **Identify the correct wordlist** by looking for the one that references movies/tv-shows.

3. **Brute-force using dictionary attack**

```
for word in wordlist:
if sha256(word) == target_hash:
print(word)
```

4. **Login as gabibboreal** by using the found password.

5. **Retrieve the flag** by opening his direct messages.

InstaFlag - Solution Template

```python
import hashlib

target_hash = "..."   #insert password hash here

with open("wordlist3.txt") as f:
    for word in f.read().splitlines():
        if hashlib.sha256(word.encode()).hexdigest() == target_hash:
            print(f"Password found: {word}")
            break
```

R0ckY0urGr4d3

Auth Challenge 03

R0ckY0urGr4d3 - Reconnaissance

R0ckY0urGr4d3 (500 points) auth

Try1ng 3v3ry p455w0rd w0n't b3 3n0ugh th15 t1m3

https://auth03.webhack.it

Flag

Submit

What can we observe? The challenge...

- provides a LUISS-themed web interface which displays the students-grading dashboard of Prof. Ralli.
- displays a sha-256 hashed action password and a wordlist on the sidebar.
- includes a button that enables changing the grade of **You** 👈 to 30/30.

R0ckY0urGr4d3 - Hypothesis

> The description of the challenge contains the warning *Trying every password won't be enough this time* in leetspeak...

Leet mapping:

- a → 4
- e → 3
- i → 1
- o → 0
- s → 5
- t → 7

R0ckY0urGr4d3 - Exploitation

1. **Retrieve the SHA-256 hash** of the action password.

2. **Download the wordlist** displayed in the web page.

3. **Apply leetspeak transformation and hash each candidate**:

```
candidate = word.translate(str.maketrans('aeiost', '431057'))
if sha256(candidate) == target_hash:
print(candidate)
```

4. **Submit the recovered password** to set the grade to 30 and retrieve the flag.

R0ckY0urGr4d3 - Solution Template

```python
import hashlib

target = "..."  #put hashed action password here

for word in open("wordlist.txt"):
    for word in f.read().splitlines():
        variation = word
        variation = variation.replace("a", "4")
        variation = variation.replace("e", "3")
        variation = variation.replace("i", "1")
        variation = variation.replace("o", "0")
        variation = variation.replace("s", "5")
        variation = variation.replace("t", "7")
        if hashlib.sha256(variation.encode()).hexdigest() == target:
            print(f"Found: {candidate}")
            break
```